

Python Tip: `for...else`

Do you need to run code after a `for` loop, but only if the loop reaches the end?

Python has a special pattern for this: `for...else`.

Let's see how it works and when it helps!



The Flag Pattern

Imagine we want to write a script that checks whether a list contains **at least one** odd number.

We can use a for loop with a True/False variable, or **flag**, called **found_odd** to remember whether the list contains an odd number. We can use **break** to stop checking as soon as we find one, since checking the rest would waste time.

```
numbers = [2, 4, 7, 8]
found_odd = False

for number in numbers:
    if number % 2 != 0:
        found_odd = True
        break

if found_odd:
    print("Found an odd number")
else:
    print("All numbers are even")
```

found_odd starts as **False**. The loop changes it to **True** if it finds an odd number. The final **if** uses that value to decide what to print.

A Shorter Way

The flag version works, but it needs an extra variable and another `if` statement after the loop.

`for...else` handles the same two outcomes with less code.

```
for item in items:
    if stop_condition:
        break
else:
    print("The loop reached the end")
```

The `else` lines up with `for`, so it belongs to the loop. If `break` stops the loop, Python skips the `else`. If the loop reaches the end without `break`, Python runs the `else`.

Case 1: The Loop Stops Early

Now let's use this pattern for our odd-number check with a list that contains an odd number.

```
numbers = [2, 4, 7, 8]

for number in numbers:
    if number % 2 != 0:
        print("Found an odd number")
        break
    else:
        print("All numbers are even")
```

Found an odd number

The loop finds **7**, so the first **print()** runs. Then **break** stops the loop, so Python skips the **else**. The second **print()** never runs.

Case 2: The Loop Finishes

Now let's look at a list with no odd numbers. Since there is no odd number, the loop never reaches `break`.

```
numbers = [2, 4, 6, 8]

for number in numbers:
    if number % 2 != 0:
        print("Found an odd number")
        break
    else:
        print("All numbers are even")
```

All numbers are even

The loop checks every number and reaches the end. Python then runs the `else`, so the second `print()` says that all the numbers are even.

One Last Note

`for...else` makes this pattern shorter, but many Python developers have not seen it before.

If you are writing code for a broad community or for newer developers, the longer `found_odd` version may be easier to understand.

Use the version that will be clearest to the people reading your code.

Wrap-Up

Use `for...else` when you want to run code only if a loop reaches the end.

If `break` stops the loop, Python skips the `else`.

If the loop reaches the end without `break`, Python runs the `else`.

It can make this pattern shorter, but the clearest code is not always the shortest code.

Follow me for more tips.

- Shep4